

UNIVERSIDAD DE GUADALAJARA



CENTRO UNIVERSITARIO DE CIENCIAS EXACTAS E INGENIERÍAS

Visualizador de Métodos de Ordenamiento

Materia: Análisis De Algoritmos

Sección: D01

Profesor: Jorge Ernesto López Arce Delgado

Alumnos: Ismael Gándara Cornejo – Angel Miguel Villalvazo Vazquez

Carrera: Ingeniería en Computación

Introducción

En esta actividad creamos un visualizador de métodos de ordenamiento. Si se quiere saber cómo una computadora organiza una lista de números de menor a mayor, esta herramienta es la respuesta. En este caso utilizamos Python y una librería llamada Tkinter para crear una interfaz gráfica. La meta principal de la actividad era que pudiéramos ver, paso a paso, cómo los diferentes algoritmos de ordenamiento funcionan y van acomodando los números hasta ordenarlos completamente.

Objetivo

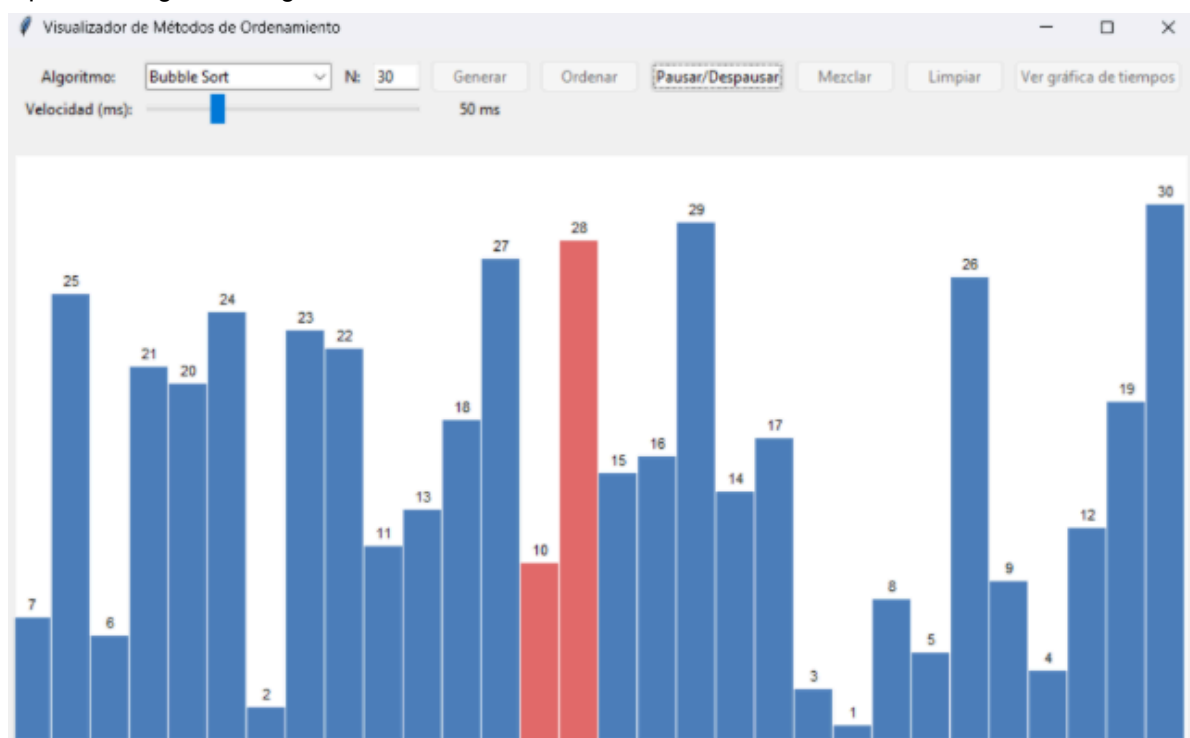
Esta actividad tiene como objetivo programar tres algoritmos de ordenamiento: bubble sort, merge sort, y quick sort aparte del selection sort que ya conocíamos. No solo tenían que funcionar, también diseñarlos de una manera que pudieran ser pausados en cualquier momento o hacerle lo que quiera el usuario. Esto de pausar se logra con una función llamada yield en Python.

Visualizador de Métodos de Ordenamiento

Para empezar, teníamos una base de código que ya incluía el Selection Sort. A partir de ahí, nos pusimos a trabajar en los nuevos algoritmos. Bubble sort fue uno de los más sencillos de implementar. Su lógica es simple, ya que recorre la lista una y otra vez, comparando cada par de elementos adyacentes y cambiándolos de lugar si están en el orden incorrecto. En cambio, merge sort es un poco más sofisticado, ya que se basa en una estrategia de "divide y vencerás". La idea es dividir la lista en dos mitades, luego cada una de esas mitades en otras dos, y así sucesivamente hasta que queden listas de un solo elemento. Para después, ir uniendo las listas de nuevo en el orden correcto. Por otra parte, quick sort, como su nombre lo dice, es uno de los más rápidos de implementar. Este elige un elemento de la lista al azar y luego reacomoda todos los demás elementos para que los más pequeños que el pivote queden a su izquierda, y los más grandes a su derecha. Después, repite el proceso con las dos sublistas de la izquierda y la de la derecha hasta que toda la lista esté ordenada.

Una vez que los algoritmos estuvieron listos, los conectamos a la interfaz donde el menú desplegable permite elegir entre los cuatro métodos. Además, los botones y los controles deslizantes funcionan como se espera, dándonos el control total sobre la visualización.

Aquí están algunas imágenes de la interfaz:



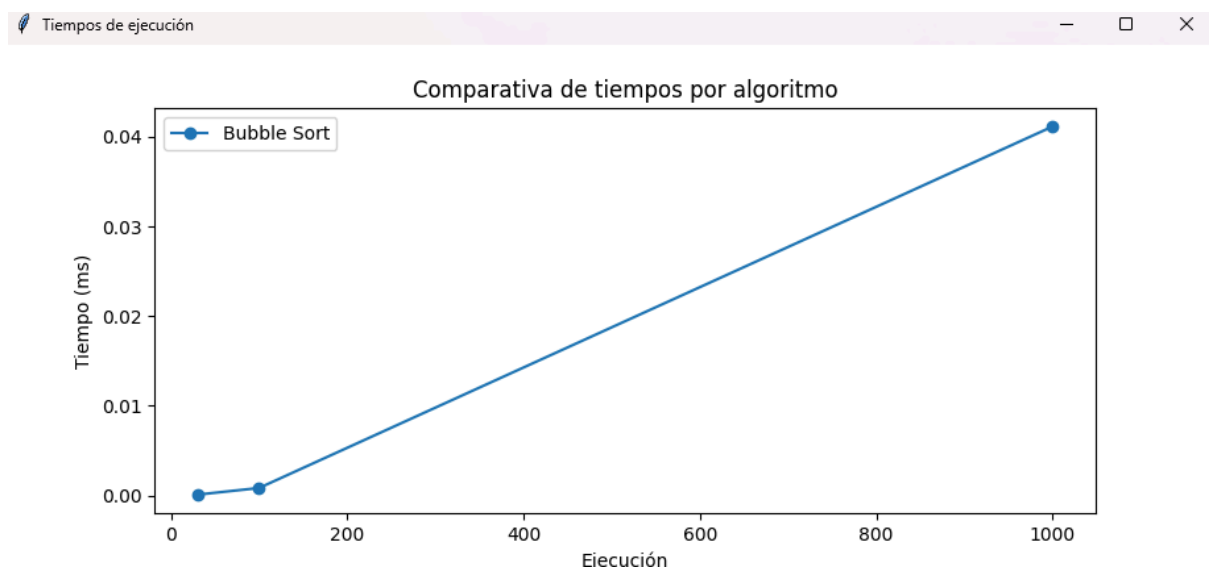
En esta imagen, pueden ver cómo las barras cambian de color para mostrar qué elementos están siendo comparados o movidos. Esto hace que el proceso sea mucho más fácil de seguir que solo viendo números en una pantalla.

Para ver cómo se comportan los algoritmos, hicimos una pequeña prueba con diferentes cantidades de barras (representando N elementos) y obtuvimos los siguientes resultados. Los tiempos que menciono son solo una estimación, basándonos en lo que tardó la animación en terminar.

Algoritmo	N = 30	N = 100	N = 1000
Selection Sort	Lento	Muy lento	Impráctico
Bubble Sort	Lento	Más Lento	Impráctico
Merge Sort	Rápido	Rápido	Rápido
Quick Sort	Rápido	Rápido	Rápido

Selection sort y bubble sort, como se observan en la tabla, son lentos con listas grandes. Se vuelven imprácticos porque el tiempo que tardan en ordenar crece de manera exponencial a medida que aumentas la cantidad de elementos. Esto se debe a que su complejidad es de $O(n^2)$. En cambio, merge sort y quick sort, con una complejidad de $O(n \log n)$, pueden manejar muchos más elementos en un tiempo razonable. Por eso, en el mundo real, cuando se necesita ordenar una base de datos o una lista muy grande, se usan algoritmos como estos que son los más eficientes.

Aquí se muestra un ejemplo de la diferencia en la gráfica de tiempos usando los datos de la tabla con un método:



Conclusión

Esta actividad fue una manera de entender, de forma práctica, las diferencias entre los algoritmos de ordenamiento. No es lo mismo leer la teoría que ver la animación en vivo, con esto es más sencillo comprender la forma de ordenamiento de cada algoritmo y cómo van separando y acomodando los datos hasta que queden en orden. Además, la herramienta que creamos es una prueba de que los algoritmos de $O(n \log n)$ son de verdad mucho más rápidos que los de $O(n^2)$ cuando el tamaño de los datos importa.

Referencias Bibliográficas

- CertiDevs. (2025, 24 enero). *Algoritmos de Ordenamiento en PseInt y Python: Burbuja, Inserción, Selección*. CertiDevs.
<https://certidevs.com/tutorial-fundamentos-programacion-algoritmos-ordenamiento>
- Dyson, X., & Dyson, X. (2024, 28 mayo). Python: Algoritmos de ordenamiento | Cable Naranja. Cable Naranja.
<https://www.cablenaranja.com/python-algoritmos-de-ordenamiento/>
- BlackeyeB. (2023, 1 mayo). Algoritmos de ordenación explicados con ejemplos en JavaScript, Python, Java y C++. freeCodeCamp.org.
<https://www.freecodecamp.org/espanol/news/algoritmos-de-ordenacion-explicados-con-ejemplos-en-javascript-python-java-y-c/>
- Algoritmo de ordenamiento. (s/f). Ecured.cu. Recuperado el 10 de septiembre de 2025, de https://www.ecured.cu/Algoritmo_de_ordenamiento